

M06 SOLO TEACHING BOOK

Self-contained teacher book | 13/13 canonical lessons

VIDEO FIRST -> close video -> SOLO TEACHING BOOK -> guided reproduction -> independent task -> validate result -> explain in your own words -> protected Review only after genuine saved attempt -> fresh changed retry if weak -> evidence/mastery.

External video can help you visualize, but required explanations, demonstrations, practice routes and validation live here too. A dead/changed video must not make M06 unusable.

DE06-01 - Verify WSL2 and choose a safe Linux workspace

Outcome

Verify or install WSL2 only when M06 begins, distinguish Windows-side WSL commands from Linux commands, and create a safe course workspace under your Linux home directory.

VIDEO FIRST

freeCodeCamp - 50 Linux & Terminal Commands | 21:05-29:43 - Windows/WSL setup and terminal orientation

[Open assigned video](#)

Current/authoritative references:

[Microsoft Learn - Install WSL](#)

SOLO TEACHING BOOK

WSL gives a Windows machine a real Linux user space. First inspect what already exists. Use PowerShell for `wsl --status` and `wsl -l -v`; use the Ubuntu/WSL terminal for `whoami`, `pwd`, `uname` and normal Linux work. Do not reinstall a working distro just because a lesson mentions installation.

Working demonstration

```
PowerShell: wsl --status; wsl -l -v
Ubuntu: whoami; pwd; uname -a; mkdir -p ~/de-course/m06
```

Guided reproduction

Verify your current WSL state, identify the distro and WSL version, enter the distro, then create `~/de-course/m06/lesson_lab`.

Independent task

Close the video/book. Prove which terminal you are in, show your Linux home path, create `~/de-course/m06/recheck`, and explain why re-running installation is unnecessary when WSL is already healthy.

Validate result

A usable distro is visible; Linux commands run inside it; the workspace is under Linux home; your note distinguishes PowerShell/WSL-management commands from Linux commands.

Explain in your own words

Explain Verify WSL2 and choose a safe Linux workspace in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

Recover if weak

If `wsl --install` shows help, inspect existing distros first. Use current Microsoft troubleshooting instead of random manual feature changes.

Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

DE06-02 - Linux navigation and safe file operations

Outcome

Navigate absolute and relative paths and create, copy, move and remove only explicitly identified sandbox files.

VIDEO FIRST

freeCodeCamp - 50 Linux & Terminal Commands | 39:05-1:27:51 - pwd, ls, cd, mkdir, touch, rm, mv, cp

[Open assigned video](#)

Current/authoritative references:

[GNU Coreutils manual](#)

SOLO TEACHING BOOK

Your current directory changes the meaning of every relative path. Before a destructive file command, establish location and target with pwd and ls. An absolute path begins at /; a relative path begins from the current working directory. cp copies, mv relocates/renames, and rm removes - none of them knows your intent.

Working demonstration

```
cd ~/de-course/m06/lesson_lab
pwd
mkdir -p sandbox/input sandbox/output
touch sandbox/input/note.txt
cp sandbox/input/note.txt sandbox/output/copy.txt
mv sandbox/output/copy.txt sandbox/output/moved.txt
ls -la sandbox/output
rm sandbox/output/moved.txt
```

Guided reproduction

Build the sandbox tree, inspect source and destination before every cp/mv/rm, and narrate exactly what changes.

Independent task

Build a different nested tree. Move one named text file through two directories; before deleting it, save pwd and ls evidence for the exact target directory.

Validate result

Only the intended sandbox object changes and you can explain absolute vs relative paths plus source vs destination.

Explain in your own words

Explain Linux navigation and safe file operations in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

Recover if weak

If location or target is uncertain, stop and run pwd plus ls -la. Never use rm -rf to resolve uncertainty.

Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

DE06-03 - Pipes, redirection, history and text inspection

Outcome

Use pipes, redirection, history and small inspection commands to understand data and command output before writing more code.

VIDEO FIRST

freeCodeCamp - 50 Linux & Terminal Commands | 1:31:56-2:44:04 - head/tail, redirection, cat/less, wc, pipes, sort/uniq, find/grep, history

[Open assigned video](#)

Current/authoritative references:

[GNU Bash - Redirections](#)

[GNU Coreutils manual](#)

SOLO TEACHING BOOK

A pipe sends standard output from one command to the next. > replaces a file; >> appends. head, tail, grep, wc, sort and uniq are small inspection tools that let you test assumptions cheaply. history helps reconstruct what was actually run.

Working demonstration

```
head data/raw/orders.csv
grep 'Completed' data/raw/orders.csv | wc -l
printf 'east\nwest\nwest\n' | sort | uniq -c
python3 src/pipeline.py > evidence.txt
echo $?
echo 'second note' >> evidence.txt
history | tail
```

Guided reproduction

Count a known status with grep|wc, create a frequency count with sort|uniq -c, redirect pipeline output, inspect the exit code, then append a second line without erasing the first.

Independent task

Use a different status/value from the bundled CSV to build one pipeline and save the result to evidence/inspection.txt.

Validate result

You can predict where output goes, demonstrate > versus >>, and break a wrong pipeline into stages to find the defect.

Explain in your own words

Explain Pipes, redirection, history and text inspection in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

Recover if weak

Run each stage separately. Do not add more pipes while you are unsure which stage changed the result.

Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

DE06-04 - Permissions, packages, sudo and command safety

Outcome

Use permissions, package commands and sudo with least privilege and reject dangerous shortcuts.

VIDEO FIRST

freeCodeCamp - 50 Linux & Terminal Commands | 4:03:47-5:00:17 - su/sudo, ownership, permissions and chmod

[Open assigned video](#)

Current/authoritative references:

[Microsoft Learn - WSL file permissions](#)

[Ubuntu Server - package management](#)

SOLO TEACHING BOOK

sudo raises privilege for one command; it is not a generic make-it-work prefix. chmod changes permission bits. 777 grants broad access and is not a normal repair. Package installation and running your own project are different privilege boundaries. On WSL, Windows-mounted files can behave differently from files inside the Linux filesystem.

Working demonstration

```
ls -l scripts/run_pipeline.sh
chmod u+x scripts/run_pipeline.sh
ls -l scripts/run_pipeline.sh
# system package metadata only when needed:
sudo apt update
```

Guided reproduction

Inspect permissions, add execute permission only to the bundled training script, and explain why sudo python3 src/pipeline.py is suspicious for a normal user-owned project.

Independent task

Classify the bundled command-safety worksheet as SAFE / NEEDS EXPLANATION / REJECT and justify target, privilege and consequence.

Validate result

Before executing a privileged/destructive command you can name the target and expected change; no chmod 777, blind recursive deletion or sudo-as-default is used.

Explain in your own words

Explain Permissions, packages, sudo and command safety in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

Recover if weak

Permission denied? Inspect owner, path and permission bits first. Do not jump straight to sudo or chmod 777.

Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

DE06-05 - Git state: init, status, add, commit, diff and log

Outcome

Operate Git deliberately across working tree, staging area and committed history while inspecting state before changing it.

VIDEO FIRST

freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 14:41-16:04 init; 20:20-24:30 status/add; 30:22-32:00 commit; 39:50-40:58 log; 54:26-55:50 diff

[Open assigned video](#)

Current/authoritative references:

[Git reference](#)

SOLO TEACHING BOOK

Git has distinct states. Editing changes the working tree; git add proposes content for the next snapshot; git commit records the staged snapshot. status and diff are observation commands. A clean history is built by understanding exactly what is staged, not by memorizing add/commit as a pair.

Working demonstration

```
git init
git status
git add README.md
git diff --staged
git commit -m "docs: establish project purpose"
git log --oneline --decorate -5
```

Guided reproduction

Initialize the lesson repository, make two focused changes, stage only one, inspect both unstaged and staged diffs, commit, then inspect history.

Independent task

Create three simultaneous states: one committed file, one staged change and one unstaged change. Explain each line of git status without changing state while explaining.

Validate result

You can point to the working-tree, index and repository states and use diff/log evidence to prove what will or did enter a commit.

Explain in your own words

Explain Git state: init, status, add, commit, diff and log in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

Recover if weak

If you are unsure what is staged, do not commit. Use git status, git diff and git diff --staged first.

Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

DE06-06 - Branches, merges and real conflict resolution

Outcome

Create focused branches, merge work and resolve a real text conflict while preserving intended content.

VIDEO FIRST

freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 40:58-52:13 - branching, checkout, merge and conflict resolution

[Open assigned video](#)

Current/authoritative references:

[Git merge documentation](#)

[Pro Git - Basic Branching and Merging](#)

SOLO TEACHING BOOK

A branch is a movable name pointing into commit history. A merge combines histories. A conflict is not Git failing; it is Git refusing to guess between incompatible edits. Resolve the actual file content, remove conflict markers, stage the resolution, commit, and prove the worktree is clean.

Working demonstration

```
git switch -c feature/docs
# edit and commit
git switch main
# conflicting edit and commit
git merge feature/docs
# resolve file intentionally
git add docs/architecture.md
git commit
git status
```

Guided reproduction

Use the bundled conflict fixture to create two incompatible edits to the same line, merge them, resolve to the required combined meaning, and record the final graph.

Independent task

Create a fresh conflict in a different file. Resolve it without deleting unrelated content and explain why keeping only one side blindly is unsafe.

Validate result

No conflict markers remain; intended content from both changes is preserved where required; git status is clean; history shows the merge.

Explain in your own words

Explain Branches, merges and real conflict resolution in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

Recover if weak

Use git status to list unmerged paths and inspect the file. If confused, stop before destructive reset; the lesson review explains safe abort/retry options.

Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

DE06-07 - GitHub remotes: clone, push, fetch, pull and tracking

Outcome

Explain and use local vs remote repositories, tracking branches, clone, push, fetch and pull without confusing network synchronization with commits.

VIDEO FIRST

freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 16:04-20:20 - remote repo/clone; 55:50-1:00:24 - push, fetch and pull

[Open assigned video](#)

Current/authoritative references:

[GitHub Docs - remote repositories](#)

[Git fetch docs](#)

SOLO TEACHING BOOK

A remote name such as origin is a local configuration pointing to another repository. push sends commits/refs; fetch downloads remote state without merging it; pull fetches and then integrates according to configuration. A local commit exists before GitHub sees it.

Working demonstration

```
git remote -v
git push -u origin main
git fetch origin
git branch -vv
git pull --ff-only
```

Guided reproduction

Use the bundled local bare remote first: push main, clone it into a second directory, make a remote-side commit, fetch from the first clone, inspect the remote-tracking branch, then integrate deliberately.

Independent task

Repeat with a changed remote update. Before pull, prove what fetch changed and what it did not change.

Validate result

You can distinguish local main, origin/main and the remote repository and explain push vs fetch vs pull.

Explain in your own words

Explain GitHub remotes: clone, push, fetch, pull and tracking in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

Recover if weak

If histories diverge, inspect git status, branch -vv and log --all --graph before choosing merge/rebase/reset.

Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

DE06-08 - Pull requests and review workflow

Outcome

Use a feature branch and pull request as a review contract, then synchronize local main after merge.

VIDEO FIRST

freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 1:16:48-1:19:57 - pull requests and collaboration

[Open assigned video](#)

Current/authoritative references:

[GitHub Docs - pull requests](#)

[GitHub Docs - creating a pull request](#)

SOLO TEACHING BOOK

A pull request is a proposed branch-to-branch change with a diff, discussion and review history. The PR does not replace Git commits. A good PR states what changed, why, how it was validated and any remaining risk. After merge, your local main does not update magically; fetch/pull it deliberately.

Working demonstration

```
git switch -c feature/readme
# edit/test/commit/push
# On GitHub: open PR -> inspect Files changed -> merge
git switch main
git pull --ff-only
```

Guided reproduction

Prepare a small branch, write a PR-style summary from the diff, and use a local review checklist. If a GitHub account is available, perform the real PR; otherwise save the exact handoff evidence for later account smoke.

Independent task

Create a second tiny branch and produce a reviewable PR packet without copying the first summary.

Validate result

Diff matches the stated scope; validation is recorded; after merge/simulation local main contains the intended commit and the feature branch is no longer the active work location.

Explain in your own words

Explain Pull requests and review workflow in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

Recover if weak

If the PR contains unrelated files, do not explain them away. Repair the branch/commits so the diff itself becomes reviewable.

Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

DE06-09 - Python virtual environments inside a repository

Outcome

Create a disposable Python virtual environment in the repository workspace while keeping it out of version control.

VIDEO FIRST

Corey Schafer - Python VENV (Mac & Linux) | Full focused venv tutorial - create, activate, use and remove/recreate

[Open assigned video](#)

Current/authoritative references:

[Python 3.14.7 - venv documentation](#)

SOLO TEACHING BOOK

A virtual environment isolates a project interpreter/package set. It is disposable, not a deliverable. The durable project contains source code and dependency/setup instructions; .venv itself should not be committed. Activation changes command lookup, but scripts can also be run with the environment interpreter directly.

Working demonstration

```
python3 -m venv .venv
source .venv/bin/activate
python -c "import sys; print(sys.executable)"
deactivate
git status --ignored --short
```

Guided reproduction

Create .venv, activate it, prove the Python path is inside the environment, deactivate it and verify Git ignores the directory.

Independent task

Delete only the disposable .venv, recreate it from saved instructions and rerun the bundled pipeline.

Validate result

Project behavior survives environment recreation and Git does not track .venv.

Explain in your own words

Explain Python virtual environments inside a repository in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

Recover if weak

If activation/path is confusing, inspect which python/python3 and sys.executable. Never solve it by committing the environment directory.

Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

DE06-11 - restore, revert, reflog and safe history repair

Outcome

Choose an appropriate Git recovery method for working-tree mistakes, committed mistakes and lost references without hiding shared history.

VIDEO FIRST

freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 1:00:24-1:03:16 restore; 1:09:47-1:12:18 revert; book adds reflog

[Open assigned video](#)

Current/authoritative references:

[git restore](#)

[git revert](#)

[git reflog](#)

SOLO TEACHING BOOK

restore operates on working-tree/index content; revert creates a new commit that inverses an earlier commit, making it suitable for shared history; reflog records local reference movements and can help recover a commit name you lost. reset can be useful, but destructive forms are not a default recovery strategy in this course.

Working demonstration

```
git restore path/to/file
git revert <bad-commit>
git reflog --oneline -10
git show <recovered-commit>
```

Guided reproduction

Make and restore an unstaged mistake, revert a committed mistake, then move a branch reference in a disposable lab and locate the prior commit via reflog.

Independent task

Given three changed incidents, select restore/revert/reflog and justify the choice before running anything.

Validate result

Final content is correct, history is explainable, and no shared-history rewrite is used as a shortcut.

Explain in your own words

Explain restore, revert, reflog and safe history repair in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

Recover if weak

Before any recovery command, classify the mistake: uncommitted content, staged content, committed/shared history, or lost reference. Then choose the smallest safe tool.

Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

DE06-12 - Shell scripts, exit codes and environment-driven commands

Outcome

Write a small Bash runner with explicit environment-driven inputs, quoting and meaningful exit codes.

VIDEO FIRST

freeCodeCamp - Bash Scripting Tutorial for Beginners | 06:21-16:23 scripts/variables/arguments; 16:23-23:23 redirection; 23:23-28:37 conditions; 41:31-42:30 exit codes

[Open assigned video](#)

Current/authoritative references:

[GNU Bash Reference Manual](#)

SOLO TEACHING BOOK

A shell script turns a command sequence into a repeatable interface. Quote variable expansions, validate required inputs before use, return exit code 0 only for success and nonzero for expected failure states. Environment variables are configuration inputs, not a reason to embed secrets in the script.

Working demonstration

```
#!/usr/bin/env bash
set -u
INPUT=${INPUT_FILE:-data/raw/orders.csv}
[[ -f "$INPUT" ]] || { echo "missing: $INPUT" >&2; exit 2; }
python3 src/pipeline.py "$INPUT"
rc=$?
exit "$rc"
```

Guided reproduction

Run the bundled script with its default input, then with INPUT_FILE set to a valid alternative and a missing file. Record stdout/stderr and exit codes.

Independent task

Write a changed runner for the fresh-retest workspace with one required environment variable and two distinct nonzero failure codes.

Validate result

Success returns 0; missing/invalid inputs return documented nonzero codes; paths containing spaces are safely quoted; no credential value is printed.

Explain in your own words

Explain Shell scripts, exit codes and environment-driven commands in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

Recover if weak

If behavior differs between interactive command and script, print only non-secret configuration/path state and inspect exit codes step-by-step.

Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.

DE06-13 - Repository handoff and clean-clone reproducibility

Outcome

Prove that another machine/user can recreate and run the project from a clean clone without hidden local state.

VIDEO FIRST

freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] | 18:05-20:20 clone + 55:50-1:00:24 remote synchronization; book adds clean-clone acceptance test

[Open assigned video](#)

Current/authoritative references:

[Git clone documentation](#)

[GitHub Docs - remote repositories](#)

[Python venv documentation](#)

SOLO TEACHING BOOK

A project is not reproducible because it works in the author directory. A clean-clone test deliberately starts without untracked files, local virtual environments or shell history. The repository must contain README/setup instructions, source, dependency/config examples and deterministic validation commands.

Working demonstration

```
git clone <remote> clean-check
cd clean-check
python3 -m venv .venv
source .venv/bin/activate
./scripts/run_pipeline.sh
git status --short
```

Guided reproduction

Push/prepare the bundled repository, clone it to a new directory, recreate its environment, run the pipeline and prove the working tree remains clean except for explicitly ignored outputs.

Independent task

Hand the changed-retest repository to a clean directory and reproduce it using only committed instructions. Record every missing assumption you discover and repair the repository rather than the clone by hand.

Validate result

A clean clone can be set up and run from documented files; no hidden absolute path or secret is required; validation totals match; repository state is explainable.

Explain in your own words

Explain Repository handoff and clean-clone reproducibility in 60-90 seconds: starting state, action, evidence, one failure mode, and how you would recover.

Recover if weak

If the clean clone fails, treat that as a project defect. Identify the hidden dependency, add a safe reproducible instruction/configuration, commit it, clone again and retest.

Mastery is competency-based, never inferred from a percentage. Save the first attempt. If materially weak, open only the matching protected Review, perform targeted repair, close it, then complete the assigned changed retry. The failed attempt stays in history.